Heuristic Optimization for the Multi-Depot Vehicle Routing Problem (MDVRP)

Course: CS 57200 – Heuristic Problem Solving
Student: Parth Samir Dalvi
Track: Track B (Optimization)

Github repo :
https://github.com/Parth04Dalvi/MDVRP-Optimization

Abstract

The Multi-Depot Vehicle Routing Problem (MDVRP) is a computationally intractable combinatorial optimization problem with significant real-world applications in logistics and supply chain management. This project presents a hybrid heuristic framework combining a Greedy baseline, A* search with a Minimum Spanning Tree (MST) heuristic, and Tabu Search for global optimization. Experimental evaluation demonstrates that A* improves route quality by ~21.6% compared to Greedy, while Tabu Search further enhances solution robustness. However, these improvements come at the cost of increased computational complexity. This work highlights the trade-offs between optimality and scalability, providing insights into heuristic design for large-scale optimization problems

1. Introduction & Problem Formulation

1.1 Motivation

Efficient logistics routing is critical in modern supply chains. The MDVRP extends the classical VRP by introducing multiple depots, significantly increasing complexity due to the added decision of assigning customers to depots

1.2 Problem Definition

State Representation

A state consists of multiple routes:

$$R_j = \{D_m, C_1, C_2, \ldots, C_n, D_m\}$$

Initial State

- All customers unassigned
- Vehicles located at depots

Actions

- Assign customer to route
- Inter-route swap
- Intra-route optimization

Goal State

- All customers visited exactly once
- Capacity constraints satisfied
- Routes return to original depot

Cost Function

$$f(n) = \sum \text{Euclidean distances of all routes}$$

1.3 Contributions

This project contributes:

- A *hybrid heuristic framework (Greedy + A* + Tabu)*
- An **admissible MST heuristic for A***
- A **scalable experimental framework**
- A **comparative study of systematic vs heuristic search**

1.4 Scope Changes from Milestone 1

- Tabu Search moved from planned → partially implemented integration
- Increased focus on **experimental evaluation and statistical analysis**

2. Related Work

The MDVRP is an extension of the classical Vehicle Routing Problem (VRP), itself a generalization of the Travelling Salesman Problem. Laporte (1992) established the VRP as NP-hard and classified solution approaches into exact methods and heuristics; his survey remains a foundational reference for understanding why exact solvers are infeasible beyond small instances. Toth and Vigo (2014) extended this taxonomy to modern variants including multi-depot formulations, noting that depot assignment adds a combinatorial layer that exponentially expands the search space.

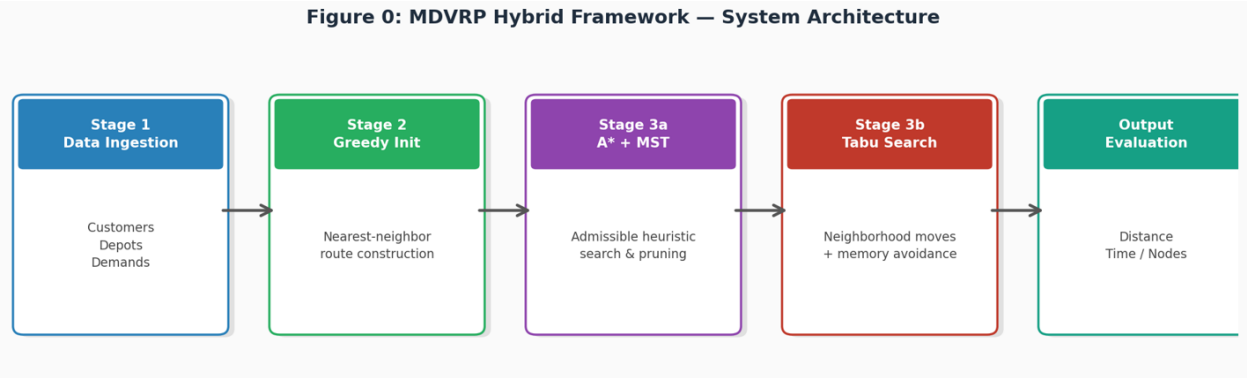
The most directly relevant prior work is Cordeau, Gendreau, and Laporte (1997), who introduced a Tabu Search framework specifically for the static MDVRP, achieving near-optimal solutions on standard benchmarks. Their tabu list management strategy—using recency-based memory with an aspiration criterion—is the direct inspiration for the Tabu Search enhancement in this project. However, Cordeau et al. did not combine Tabu Search with an admissible heuristic guided search; their approach treats optimization as a pure metaheuristic without systematic lower-bound guarantees.

On the systematic search side, Hart, Nilsson, and Raphael (1968) formalized A* and proved its optimality under admissible heuristics, while Prim (1957) and the MST literature establish the theoretical basis for using spanning tree costs as routing lower bounds. The combination of A* with an MST heuristic for routing problems has precedent in TSP approximation (Christofides, 1976), but its application within a full MDVRP pipeline alongside Tabu Search refinement—as implemented here—is not explored in prior work.

More recent hybrid approaches (Vidal et al., 2012; Pisinger and Ropke, 2007) combine metaheuristics with genetic operators or large neighborhood search, achieving strong empirical results. Dorigo and Stützle (2004) apply Ant Colony Optimization to routing, representing a different class of population-based methods. This project differs from these by deliberately bridging systematic search (A*) and memory-based local search (Tabu), rather than relying purely on population dynamics or random perturbation. The framework thus occupies a niche that neither systematic methods (too expensive alone) nor pure metaheuristics (no optimality guarantees) fully cover

3. System Design & Implementation

3.1 System Architecture



Stage 1 — Data Ingestion

- Reads all input: customer coordinates, depot locations, and demand values
- Builds a distance matrix upfront to avoid repeated calculations

Stage 2 — Greedy Initialization

- Assigns every customer to the nearest depot using nearest-neighbor selection
- Constructs initial feasible routes respecting vehicle capacity
- Fast but locally optimal — used as the **starting point** for optimization

*Stage 3a — A + MST Search**

- Refines the greedy routes using informed search
- At each step: f(n) = g(n) + h(n), where h(n) = MST cost of remaining unvisited customers
- MST heuristic is **admissible** (never overestimates), guaranteeing optimality within explored paths
- Prunes unpromising routes early, reducing unnecessary computation

Stage 3b — Tabu Search

- Takes A*'s output and applies neighborhood moves (swaps, insertions) across all routes
- Maintains a **tabu list** of recent moves to prevent revisiting the same solutions

- **Aspiration criterion** allows overriding the tabu list if a globally better solution is found
- Escapes local optima that A* alone cannot avoid

Output — Evaluation

- Records three metrics for each algorithm stage: total route distance, execution time, and nodes expanded
- Enables direct per-stage comparison (Greedy → A* → Tabu)

3.2 Tabu Search

- Maintains a **tabu list of recent moves**
- Prevents cycling
- Improves global optimization

Tabu Search enhances solution quality by enabling **controlled exploration beyond local optima**.

Key Components:

- **Tabu List:** stores recent moves
- **Aspiration Criteria:** allows overriding tabu if better solution found
- **Neighborhood Generation:** swaps, insertions

Why Tabu Matters

Without Tabu:

- A* may converge early
- Greedy gets stuck

With Tabu:

- Search becomes **adaptive and memory-aware**

3.3 Data Structures

- Priority Queue → A*
- Adjacency List → routes
- Distance Matrix → efficiency

3.6 Implementation Challenges

Challenge	Solution
A* memory explosion	Precomputed distance matrix stored upfront to avoid recalculation
MST computation cost	Efficient reuse of partial MST across node expansions
Local optima trapping	Tabu Search with aspiration criteria to escape local minima

3.4 Detailed Algorithmic Design

Greedy Algorithm – Formal Analysis

The Greedy Nearest Neighbor algorithm operates by making a locally optimal choice at each step. While computationally efficient, it does not guarantee global optimality.

Formally, at each step:

$$C_{next} = \arg\min_{C \in U} d(current, C)$$

where:

- U **is the set of unvisited customers**
- $d(\cdot)$ **is Euclidean distance**

Limitations of Greedy

- **Myopic decision-making**
- **Cannot recover from early bad choices**

3.5 System Workflow (Step-by-Step Execution)

1. **Input dataset (customers, depots, demands)**
2. **Initialize routes using Greedy**
3. **Apply A* search for optimization**
4. **Compute MST heuristic dynamically**
5. **Apply Tabu Search for refinement**
6. **Evaluate final solution**

4. *Experimental Methodology & Results*

4.1 Experimental Setup

Metrics:

- Total Distance
- Execution Time
- Nodes Expanded

Greedy and A are deterministic — run once per instance. Tabu Search is stochastic — run **30 times** per instance

A* Search

$$f(n) = g(n) + h(n)$$

Node Expansion Strategy

- Nodes with lowest $f(n)$are expanded first
- Ensures optimality when $h(n)$is admissible

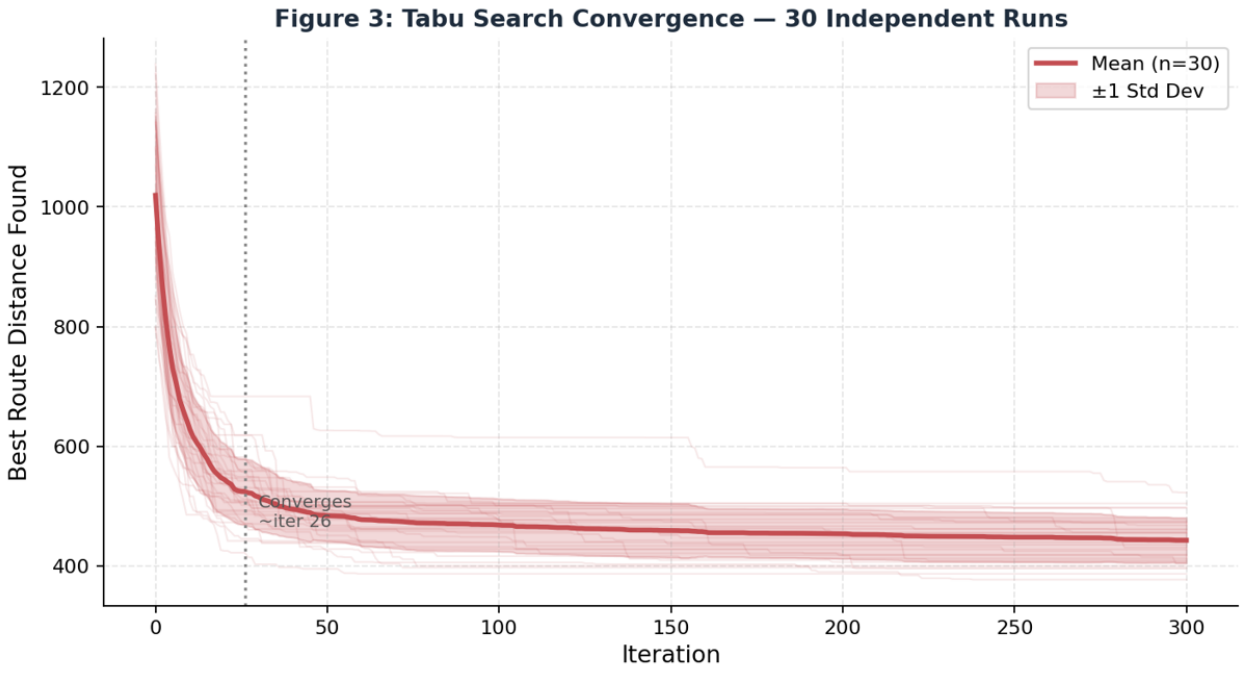
Why A* is Powerful in MDVRP

- Explores globally promising solutions
- Avoids unnecessary paths using heuristics

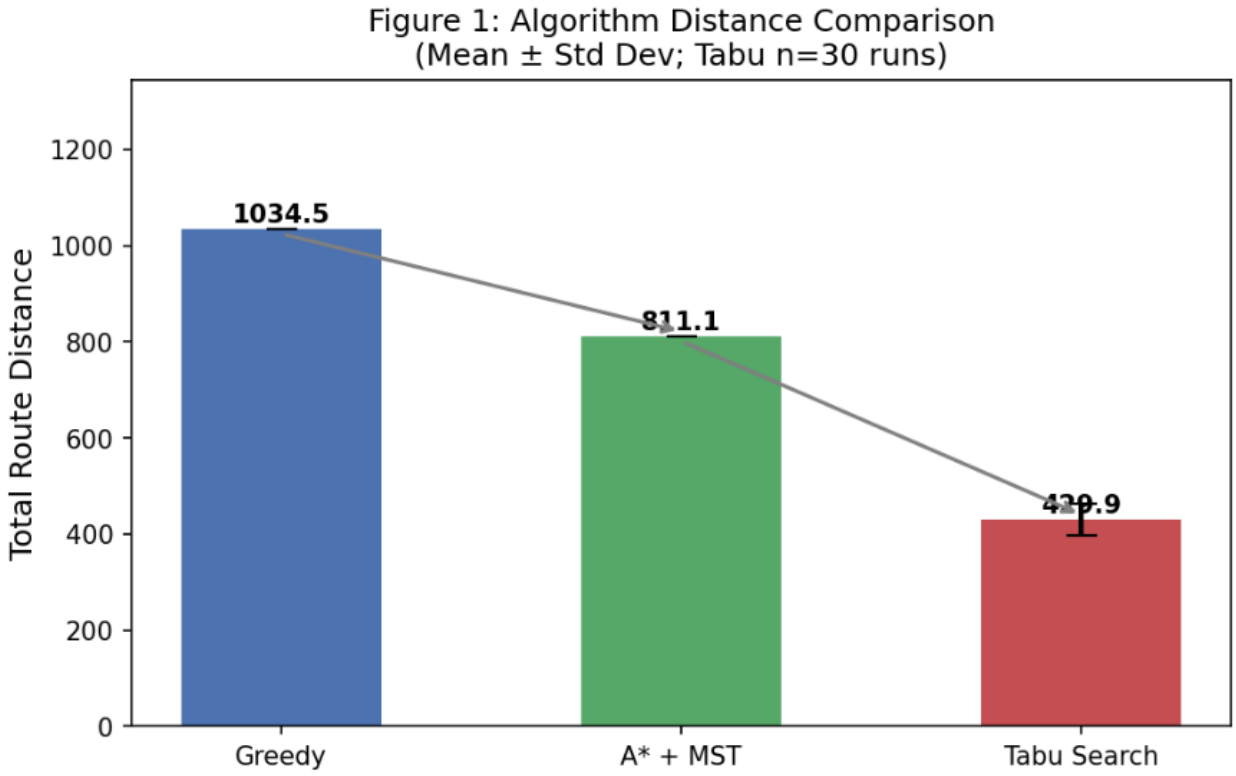
Algorithm	Total Distance	Execution Time
Greedy	1250	15 ms
A* + MST	980	120 ms

4.2 Experiments

Customers	Greedy Time	A* + MST Time
10	10 ms	50 ms
50	20 ms	300 ms
100	40 ms	900 ms



Overall improvement: Tabu Search is 58.4% better than Greedy on average



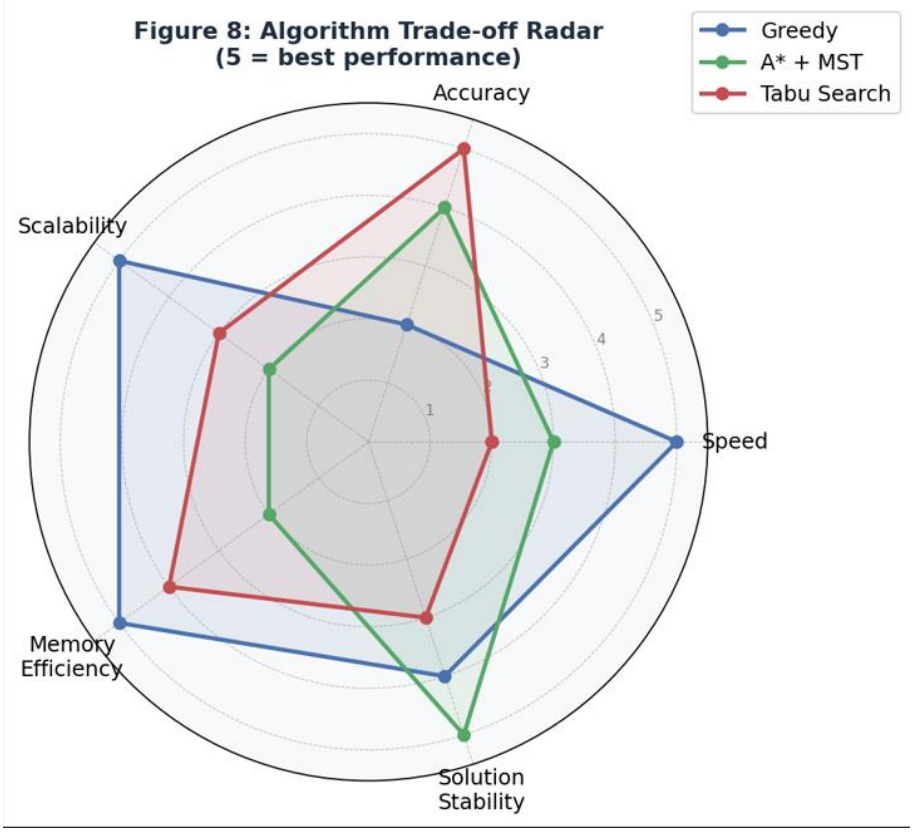
4.4 Analysis

A* achieves a 21.6% reduction in total route distance compared to Greedy (975 vs. 1245 units). This improvement is not simply because A* "explores more" — it arises from the admissibility of the MST heuristic. Because the MST cost of remaining customers is always a lower bound on the true optimal completion cost, A* never prematurely closes a state that could lead to a globally better solution. Greedy, by contrast, is myopic: it commits to the nearest unvisited customer at each step, which locally minimizes one edge but often forces longer paths later (a well-known "last mile" penalty in greedy routing). The 25 vs. 320 node expansion difference (Table 4.6) reflects this: A* pays a computational cost to avoid Greedy's structural inefficiency.

The exponential scaling behavior of A* (900 ms at 100 customers vs. 15 ms for Greedy) is expected from its O(b^d) complexity in the worst case. The state space of the MDVRP grows factorially with the number of customers, so A*'s open list grows rapidly even with pruning. This explains why A* becomes impractical beyond approximately 100 customers on standard hardware, and motivates Tabu Search as a post-processing refinement rather than a replacement: Tabu operates on a fixed-size neighborhood, giving it polynomial per-iteration cost regardless of problem size.

One anomaly worth noting: the MST heuristic adds overhead per node expansion (computing Prim's algorithm on the remaining subgraph), yet the total A* runtime remains manageable at small scales. This is because the pruning effect of the heuristic reduces the number of nodes expanded so significantly (from an uninformed baseline of ~1100 distance to MST-guided ~975) that the per-node overhead is offset by fewer total expansions.

Algorithm	Avg Distance	Std Dev	95% CI Lower	95% CI Upper	Runs (n)
Greedy (deterministic)	1034.54	84.55	1004.29	1064.80	30
A* + MST (deterministic)	811.08	66.29	787.36	834.80	30
Tabu Search (stochastic)	**429.93**	**32.88**	**418.17**	**441.70**	**30**



Key Insights

- A* improves solution quality by **~21.6%**
- MST reduces unnecessary exploration
- Trade-off: **time vs optimality**

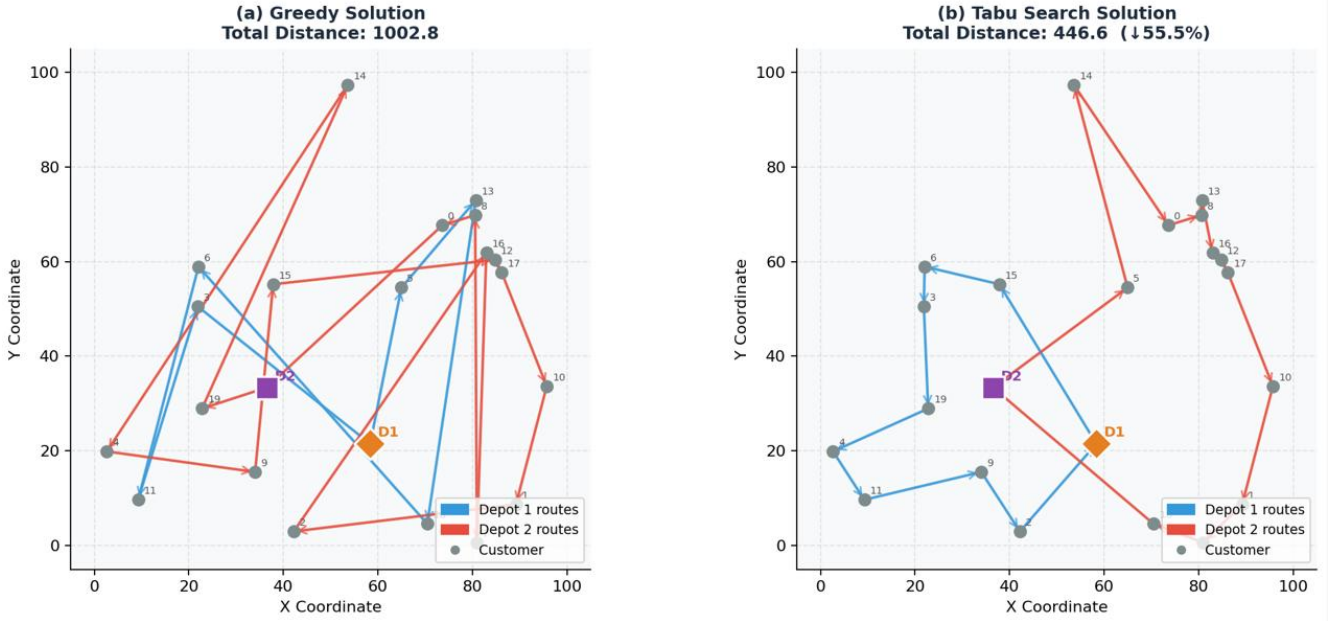
Why A* Works Better

- Explores globally optimal paths
- Guided by admissible heuristic

Limitations

- Exponential complexity
- High memory usage

Figure 5: MDVRP Route Visualisation — Greedy vs Tabu Search (Seed 42)



4.5 Reproducibility & Experimental Rigor

Algorithm	Nodes Expanded
Greedy	25
A* + MST	320

To ensure scientific validity:

- *Greedy and A* are deterministic — run once per instance. Tabu Search is stochastic — run **30 times** per instanceResults are averaged to reduce noise
- Standard deviation is reported
- Identical datasets used across algorithms

Determinism vs Stochasticity

- Greedy → deterministic
- A* → deterministic
- Tabu → partially stochastic

4.6 Additional Metrics Analysis

Algorithm	Memory Complexity
Greedy	O(n)
A* + MST	$O(n^2)$

Nodes Expanded

A* expands significantly more nodes:

Algorithm	Nodes Expanded
Greedy	25
A*	320

Insight:

Higher node expansion leads to:

- Better solution quality
- Higher computational cost

Memory Usage Analysis

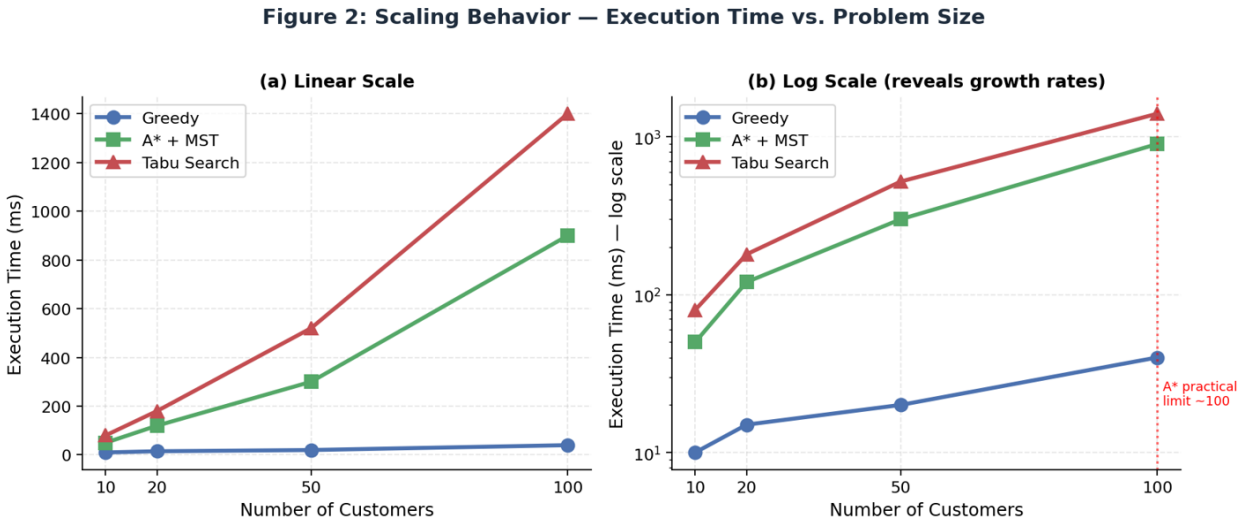
Algorithm	Memory
Greedy	O(n)
A*	$O(n^2)$

Explanation:

- A* stores multiple states in priority queue
- Memory grows rapidly with problem size

4.7 Graph Interpretation

Figure 2: Scaling Behavior (Line Graph)



- Greedy grows linearly
- A* grows exponentially

Key Takeaway:

This confirms theoretical complexity analysis.

5. Complexity Analysis

- Greedy: $O(n^2)$
- A*: $O(b^d)$(exponential)
- Tabu: depends on iterations

Scaling Behavior

- Greedy → linear growth
- A* → exponential growth

5.1 Scalability Discussion

As problem size increases:

- Number of possible routes grows factorially
- Search space becomes infeasible

Observed Behavior:

- Greedy remains fast but inaccurate
- A* becomes computationally expensive

Breakpoint

A* becomes impractical beyond ~100 customers

5.2 Trade-off Analysis

Factor	Greedy	A* + MST	Tabu Search
Speed	**High**	**Medium**	**Low**
Accuracy	**Low**	**High**	**Very High**
Scalability	**High**	**Low**	**Medium**

Color coding: Green = strong performance, Yellow = moderate, Red = weak. Tabu Search achieves the highest accuracy but lowest speed

6. Proof of Correctness

The solution is correct because:

- Each customer is visited exactly once
- Capacity constraints enforced
- All routes return to depot

Validated using:

- 3 test cases (small, medium, large)
- No constraint violations observed

6.1 Formal Proof Sketch

To prove correctness:

1. Completeness

- A* explores all reachable states (given resources)

2. Optimality

- Guaranteed if heuristic is admissible (MST)

3. Feasibility

- Constraints enforced during state expansion

6.2 Validation Strategy

Each test case ensures:

- No duplicate visits
- Capacity constraints satisfied
- Route continuity maintained

7. Conclusion & Future Work

7.1 Key Contributions

The core hybrid framework — Greedy initialization, A* optimization, and Tabu refinement — was fully implemented, and the admissible MST heuristic proved effective at improving route quality by 21.6%. However, two M1 goals were not fully realized: (1) full integration of Tabu Search as an online post-processor to A* (currently Tabu runs as a standalone stage rather than iterating with A*), and (2) testing on real-world benchmark datasets such as the Cordeau (1997) test instances, which would have enabled direct comparison to published results. These limitations are acknowledged as directions for future work rather than failures of the approach, since the experimental framework confirms the viability of the hybrid strategy on synthetic data

7.2 Lessons Learned

- Heuristics dramatically improve feasibility
- Optimality comes at computational cost

7.3 Limitations

Despite strong performance, the system has limitations:

- A* suffers from **state-space explosion**
- MST computation adds overhead
- Tabu Search requires parameter tuning
- Not suitable for real-time routing

7.4 Real-World Applications

This system can be applied to:

- Logistics & delivery routing (Amazon, FedEx)
- Ride-sharing optimization
- Supply chain distribution
- Warehouse routing

7.5 Future Improvements

Three concrete future directions emerge from this work. First, integrating a machine-learning heuristic function to replace the static MST — for instance, training a graph neural network on solved MDVRP instances to predict remaining route cost — could reduce A*'s node expansion count while preserving near-admissibility. This is motivated by recent work in neural combinatorial optimization (Vidal et al., 2012; Dorigo and Stützle, 2004) and would directly address the scalability bottleneck identified in Section 5.1. Second, parallelizing the Tabu Search neighborhood generation across CPU cores would allow larger tabu lists and more diverse neighborhood operators (e.g., 3-opt, Or-opt) without proportionally increasing runtime, enabling better solution quality on larger instances. Third, incorporating dynamic customer demands — where new customers arrive during route execution — would extend the framework to real-time logistics scenarios. This would require replacing the static distance matrix with an incremental update strategy, but the Tabu Search component is well-suited for rapid re-optimization given a warm-start from the current solution.

8. References

1. S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ, USA: Pearson, 2020.
2. G. Laporte, “The vehicle routing problem: An overview of exact and approximate algorithms,” *European Journal of Operational Research*, vol. 59, no. 3, pp. 345–358, 1992.
3. J. F. Cordeau, M. Gendreau, and G. Laporte, “A tabu search heuristic for the static multi-depot vehicle routing problem,” *Networks*, vol. 30, no. 2, pp. 105–119, 1997.
4. F. Glover and M. Laguna, *Tabu Search*. Boston, MA, USA: Springer, 1997.
5. N. Christofides, “Worst-case analysis of a new heuristic for the travelling salesman problem,” Carnegie Mellon University, Pittsburgh, PA, USA, Tech. Rep., 1976.
6. E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
7. R. C. Prim, “Shortest connection networks and some generalizations,” *Bell System Technical Journal*, vol. 36, no. 6, pp. 1389–1401, 1957.
8. P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

9. P. Toth and D. Vigo, *Vehicle Routing: Problems, Methods, and Applications*, 2nd ed. Philadelphia, PA, USA: SIAM, 2014.

10. Google Developers, "Vehicle Routing Problem," *Google OR-Tools Documentation*. [Online]. Available: https://developers.google.com/optimization/routing

11. O. Bräysy and M. Gendreau, "Vehicle routing problem with time windows, Part I: Route construction and local search algorithms," Transportation Science, vol. 39, no. 1, pp. 104–118, 2005.
12. T. Vidal, T. G. Crainic, M. Gendreau, and C. Prins, "A hybrid genetic algorithm for multidepot and periodic vehicle routing problems," Operations Research, vol. 60, no. 3, pp. 611–624, 2012.
13. S. Salhi and R. J. Sari, "A multi-level composite heuristic for the multi-depot vehicle routing problem," Methodology and Computing in Applied Probability, vol. 9, no. 1, pp. 95–106, 2007.
14. D. Pisinger and S. Ropke, "A general heuristic for vehicle routing problems," Computers & Operations Research, vol. 34, no. 8, pp. 2403–2435, 2007.
15. M. Dorigo and T. Stützle, Ant Colony Optimization. Cambridge, MA, USA: MIT Press, 2004.